Project Report: Vehicle Parking App (V1)

Student Details:

- **Name:** Nakka Rajasekhar
- **Student ID:** 22f3001169
- **Mail id** : 22f3001169@ds.study.iitm.ac.in
- **Course:** Modern Application Development I (MAD I)

Description This project develops a multi-user 4-wheeler parking application. It aims to efficiently manage parking lots and spots for administrators, while providing users with intuitive search, reservation, and history tracking functionalities. The solution is designed for robustness and ease of use.

1. Project Overview

1.1. The Challenge Addressed

Our project built a smart, multi-user 4-wheeler parking app to seamlessly manage parking lots, individual spots, and parked vehicles. It caters to both administrative oversight and everyday user needs, aiming for an efficient, intuitive, and robust solution.

1.2. Development Journey

We built the backend using **Flask** and used **SQLite** to manage our database. We clearly separated the roles for **Admins** and **Users**:

- **Admins** can add parking lots (with address, pin code, and city), manage parking spots, and see user activity.
- **Users** can sign up, log in, search by pin code or city, book or free a parking spot, and view their parking history (including total hours and cost).

1.3. AI Assistance

Approximately 15% of this project involved AI tools for assistance, primarily in generating initial code structures, debugging suggestions, and refining textual explanations for the report and presentation script.

2. Technologies Used

Our application is built upon:

- **Backend:** Flask (Python web framework).
- **Database:** SQLite3 (self-contained SQL database).
- **Frontend:** HTML5, CSS3, JavaScript.
- **Styling/UI:** Bootstrap 5.3.0 (responsive design, components).
- **Icons:** Font Awesome 6.0.0-beta3.

3. DB Schema Design

The database has four interconnected tables:

1. **users**: id (PK, AUTO), username (UNIQUE, NOT NULL), password (NOT NULL).
2. **parking_lots**: id (PK, AUTO), name (NOT NULL), price_per_hour (REAL, NOT NULL), max_spots (NOT NULL), address (NOT NULL), pin_code (NOT NULL), city (NOT NULL).
    - *Reasoning:* All fields are mandatory to ensure complete location and pricing data for management and user search.
3. **parking_spots**: id (PK, AUTO), lot_id (FK to parking_lots), status (TEXT DEFAULT 'A').
    - *Reasoning:* Links spots to their lot and tracks real-time availability.
4. **reservations**: id (PK, AUTO), user_id (FK to users), spot_id (FK to parking_spots), in_time (TEXT), out_time (TEXT), cost (REAL).
    - *Reasoning:* Captures full parking session details, including cost, linking users to specific spots and times.

4. API Design

The application utilizes Flask routes as its API resource endpoints, handling HTTP requests (GET, POST). Key interactions include:

- **Authentication:** /, /register, /login, /logout.
- **Admin Module:** /admin (dashboard), /admin/create_lot, /admin/edit_lot/<id>, /admin/delete_lot/<id>, /admin/user_details/<id>.
- **User Module:** /user (dashboard with search), /reserve_spot/<id>, /release_spot/<id>.

The API design adheres to RESTful principles for clear resource interaction. The YAML file for API definition is to be submitted separately.

Additional Features:

- **Enhanced User Search:** Users can search for parking lots by both **pin code and city name**, providing flexible location-based filtering.
- **Interactive Dashboards:** Both Admin and User dashboards feature dynamic charts (Chart.js) for visual data summaries (e.g., overall spot status for admin, total parking hours/cost for user).
- **Detailed Admin User View:** Admins have a dedicated page for each user, providing comprehensive profile and full parking reservation history.
- **Responsive UI/UX:** Frontend is designed with Bootstrap for optimal viewing across devices.

5.Presentation Video Link :
https://drive.google.com/file/d/1jH0j1PyyEtFeFDiHjCC4t_V3kiBCA65D/view?usp=sharing